
title: "My Personal Website — Complete Technical Analysis Report"
subtitle: "Software Architecture & Code Documentation"
author: "Technical Documentation Team"
date: "July 2026"

<!-- Page break for title page simulation -->

My Personal Website

Complete Technical Analysis Report

Project Version: 1.0.0

Analysis Date: July 2026

Document Type: Software Architecture & Code Documentation

This document provides a comprehensive technical analysis of the My Personal Website project, covering architecture, technology stack, source code, UI/UX, data flow, and developer documentation. It is intended for software engineering study, academic reference, developer onboarding, and portfolio case study purposes.

Table of Contents

1. Project Overview

- 1.1 Project Name & Purpose
- 1.2 Problem It Solves
- 1.3 Target Users
- 1.4 Main Features
- 1.5 Project Scope

2. Technology Stack Analysis

- 2.1 Programming Languages
- 2.2 Frameworks & Libraries
- 2.3 Dependencies
- 2.4 Build Tools
- 2.5 Database Technologies
- 2.6 APIs & External Services
- 2.7 Development Environment Requirements

3. Complete Project Structure

- 3.1 Folder Structure Diagram
- 3.2 File Organization
- 3.3 Purpose of Each Major Folder
- 3.4 Purpose of Important Files
- 3.5 Entry Points
- 3.6 Configuration Files

4. Architecture Analysis

- 4.1 Overall Application Architecture
- 4.2 Architecture Diagram
- 4.3 Design Patterns Used
- 4.4 Frontend Architecture
- 4.5 Backend Architecture
- 4.6 Data Flow
- 4.7 Data Flow Diagram
- 4.8 Component/Module Relationships
- 4.9 Communication Between Layers

5. Source Code Analysis

- 5.1 Main Application Flow
- 5.2 Routing System
- 5.3 State Management
- 5.4 API Communication
- 5.5 Data Handling
- 5.6 Error Handling
- 5.7 Authentication / Authorization
- 5.8 Important Modules Deep-Dive

6. UI/UX Analysis

- 6.1 User Interface Structure

- 6.2 Pages / Screens
- 6.3 Components
- 6.4 Design System
- 6.5 User Interactions
- 6.6 Responsive Behavior
- 6.7 Accessibility Considerations

7. Database Analysis

- 7.1 Database Structure
- 7.2 Tables / Collections
- 7.3 Relationships
- 7.4 Data Models
- 7.5 CRUD Operations
- 7.6 Data Flow

8. API Documentation

- 8.1 Track Endpoint
- 8.2 Travel Endpoint
- 8.3 Route Endpoint
- 8.4 Google Apps Script API
- 8.5 API Workflow Diagram

9. Dependencies Report

- 9.1 Production Dependencies
- 9.2 Development Dependencies

10. Feature-by-Feature Explanation

- 10.1 Home / Landing Page
- 10.2 Travel Log Interactive Map
- 10.3 Blog Reader
- 10.4 Profile Page
- 10.5 CV/Data Download
- 10.6 Visitor Tracking
- 10.7 Session Timeout Management

11. Code Quality Review

- 11.1 Code Organization
- 11.2 Maintainability
- 11.3 Scalability
- 11.4 Performance Considerations
- 11.5 Security Considerations
- 11.6 Best Practices Followed

12. Learning Guide

- 12.1 What Should Be Learned First
- 12.2 How the Project Works Internally
- 12.3 Important Files to Study First
- 12.4 Suggested Learning Order

13. Developer Documentation

- 13.1 Installation Requirements
- 13.2 Setup Explanation
- 13.3 Running Instructions
- 13.4 Environment Variables Explanation
- 13.5 Deployment Overview

14. Final Summary

- 14.1 Project Strengths
- 14.2 Architecture Summary
- 14.3 Key Technologies
- 14.4 Overall Evaluation

1. Project Overview

1.1 Project Name & Purpose

Project Name: My Personal Website

Purpose: A minimalist personal portfolio website that serves as a digital journal and interactive resume for the developer (Abilash). The site showcases the developer's personality, travel history, blog posts, and professional profile — all presented through a dark-themed, modern UI with smooth animations.

1.2 Problem It Solves

- Provides a centralized, visually appealing platform to present a personal brand
- Replaces static resumes with an interactive, dynamic digital presence
- Demonstrates technical skills through real-world implementation (interactive maps, tracking, API integration)
- Offers a downloadable data/portfolio PDF for offline reference
- Logs visitor analytics for the site owner to understand audience engagement

1.3 Target Users

- **Primary:** Abilash (the developer) — personal branding and portfolio
- **Secondary:** Recruiters, collaborators, and visitors interested in learning about the developer
- **Tertiary:** Developers studying the codebase for learning React, Vite, Leaflet, Express, and Google Apps Script integration

1.4 Main Features

#	Feature	Description
1	Home Page	Hero section with personal intro, trait badges, navigation panels, "Say Hello" greeting overlay
2	Travel Log	Interactive world map (Leaflet) with custom markers, travel routes (OSRM routing), animated paths
3	Blogs	Filterable blog list with tag-based filtering, full-screen reading modal with custom cursor
4	Profile	Personal profile card, status grid, social links, interests section
5	Data Download	Modal prompting for visitor name, triggers PDF download + tracking
6	Visitor Tracking	Collects browser, device, OS, screen, language, referrer, geolocation data via Google Sheets
7	Session Timeout	Inactivity detection (60s) with a prompt to re-engage the visitor
8	Responsive Design	Full mobile-responsive layout with hamburger navigation drawer

1.5 Project Scope

The project is a **single-page application (SPA)** powered by React with client-side routing. It uses Vite as the build tool, Tailwind CSS v4 for styling, and Framer Motion for animations. The backend is an Express.js server providing API proxying for OSRM (Open Source Routing Machine) routes, visitor tracking relay to Google Apps Script, and static file serving for the built frontend. Data persistence is handled entirely through **Google Sheets** via a **Google Apps Script** Web App — there is no traditional database.

2. Technology Stack Analysis

2.1 Programming Languages

Language	Usage
JavaScript (ES6+)	Entire frontend (React JSX) and backend (Node.js Express)
CSS	Tailwind CSS v4 utility classes + custom CSS for Leaflet overrides
HTML	Single entry point (<code>index.html</code>)
Google Apps Script	JavaScript-based serverless functions for Google Sheets integration

2.2 Frameworks & Libraries

Frontend

Library	Version	Purpose
React	^18.2.0	UI component library
React Router DOM	^6.20.0	Client-side routing
Framer Motion	^12.42.2	Animation library
Leaflet	^1.9.4	Interactive map rendering
Lucide React	^1.25.0	SVG icon library
Tailwind CSS	^4.3.3	Utility-first CSS framework

Backend

Library	Version	Purpose
Express	^4.22.2	HTTP server framework
Serve	^14.2.4	Static file serving (alternative)

Build / Dev Tools

Tool	Version	Purpose
Vite	^5.0.0	Build tool and dev server
@vitejs/plugin-react	^4.2.0	React Fast Refresh for Vite
@tailwindcss/vite	^4.3.3	Tailwind CSS Vite plugin

2.3 Dependencies

See Section 9 — **Dependencies Report** for the full table.

2.4 Build Tools

- **Vite 5** — Frontend bundler with Hot Module Replacement (HMR) during development
- **ESLint** — Code linting (configured but no config file visible in root — likely uses default or extends)
- **npm** — Package manager (with `legacy-peer-deps=true` flag in `.npmrc`)

2.5 Database Technologies

There is **no traditional database**. The project uses:

- **Google Sheets** as a document store (via Google Apps Script)
- **localStorage** (browser) for caching travel route data from OSRM
- Files on disk for the downloadable PDF (`public/Abilash-Data.pdf`)

2.6 APIs & External Services

Service	Endpoint	Purpose
Google Apps Script	<code>script.google.com/macros/s/AKfycb.../exec</code>	CV download tracking + travel data + visitor analytics
OSRM (Open Source Routing Machine)	<code>router.project-osrm.org</code>	Driving directions for travel route polylines
ipapi.co	<code>ipapi.co/{ip}/json/</code>	IP geolocation (city, country, region) for visitor tracking
CartoDB	<code>basemaps.cartocdn.com/dark_all/...</code>	Dark tile layer for the Leaflet map

2.7 Development Environment Requirements

- **Node.js** >= 18 (ESM modules, `fetch` API)
- **npm** >= 9
- Modern web browser (Chrome, Firefox, Edge, Safari)
- Text editor / IDE (VS Code recommended)
- Git (for version control)

3. Complete Project Structure

3.1 Folder Structure Diagram

```

my-personal-website/
├── .env                                # Environment variables (development)
├── .env.example                       # Environment variable template
├── .env.production                   # Production environment variables
├── .gitignore                        # Git ignore rules
├── .npmrc                            # npm config (legacy-peer-deps=true)
├── apps-script-code.gs              # Google Apps Script (simpler version)
├── apps-script.js                   # Google Apps Script (full version with travel)
├── index.html                       # HTML entry point
├── package-lock.json                # Dependency lock file
├── package.json                     # Project manifest & scripts
├── server.js                        # Express.js backend server
├── vite.config.js                   # Vite configuration
├── public/                          # Static assets (served as-is)
│   ├── Abilash-Data.pdf             # Downloadable portfolio PDF
│   └── Website-Review.pdf           # Additional PDF resource
├── dist/                            # Build output (generated)
│   ├── index.html
│   ├── Abilash-Data.pdf
│   └── assets/
│       ├── index-*.js
│       ├── index-*.css
│       └── (image files)
├── src/                             # Application source code
│   ├── main.jsx                    # React app entry point
│   ├── App.jsx                     # Root component with routing
│   ├── index.css                   # Global styles (Tailwind + custom)
│   ├── components/                 # Reusable UI components
│   │   ├── DownloadModal.jsx       # PDF download prompt modal
│   │   └── TimeoutModal.jsx        # Session timeout overlay
│   ├── layouts/                    # Layout components
│   │   └── Shell.jsx               # Main app shell (header, nav, drawer)
│   ├── pages/                      # Route page components
│   │   ├── Home.jsx               # Landing / hero page
│   │   ├── TravelLog.jsx          # Interactive map page
│   │   ├── Blogs.jsx              # Blog listing & reader
│   │   └── Profile.jsx             # Personal profile page
│   ├── services/                   # Service / utility modules
│   │   ├── download.js             # PDF download helper
│   │   ├── map.js                 # Leaflet map utilities (markers, routes, tiles)
│   │   ├── profileImg.js          # Dynamic profile image resolver
│   │   └── track.js               # Visitor & download tracking service
│   ├── images/                     # Static image assets
│   │   ├── batman.jpg
│   │   ├── bike.jpg
│   │   ├── god.webp
│   │   ├── logos.png
│   │   ├── music.jpg
│   │   ├── pet.webp
│   │   ├── profile.jpeg
│   │   └── tea.webp
│   ├── assets/                     # (empty) — reserved for other assets
│   ├── hooks/                      # (empty) — reserved for custom hooks
│   └── utils/                       # (empty) — reserved for utility functions
└── node_modules/                   # Installed packages (ignored by git)

```

3.2 File Organization

The project follows a **feature-based + role-based** hybrid structure:

- **src/pages/** — One file per route/page (Home, TravelLog, Blogs, Profile)
- **src/components/** — Reusable UI pieces (modals)
- **src/layouts/** — Shell layout wrapping all pages (header, navigation, footer)
- **src/services/** — Business logic / API interaction modules
- **src/images/** — Static images imported as modules
- **Root level** — Configuration, server, entry HTML, Google Apps Script files

3.3 Purpose of Each Major Folder

Folder	Purpose
src/	All application source code
src/pages/	Top-level route components rendered by React Router
src/components/	Reusable UI components used across multiple pages
src/layouts/	Layout wrapper component (navigation shell)

src/services/	Non-UI logic: API calls, map rendering, analytics
src/images/	Imported image assets
public/	Static files served directly by the web server
dist/	Vite build output (production build)

3.4 Purpose of Important Files

File	Purpose
index.html	HTML shell with root div, Google Fonts preconnect, and module script entry
src/main.jsx	React DOM entry point — renders App inside BrowserRouter
src/App.jsx	Root component: defines routes, manages global state (cvOpen, timedOut)
src/index.css	Tailwind CSS v4 import, custom theme (colors, fonts, animations), Leaflet overrides
server.js	Express server: API proxy for tracking, travel data, OSRM routes; static file hosting
vite.config.js	Vite configuration: React plugin, Tailwind plugin, relative base path
apps-script.js / apps-script-code.gs	Google Apps Script deployed as Web App for Sheets interaction
.env / .env.example / .env.production	Environment configuration for Google Sheets API URL

3.5 Entry Points

- **Development (Vite):** index.html → src/main.jsx → src/App.jsx
- **Production (Express):** server.js (serves dist/ static files, API routes, then SPA fallback)
- **Google Apps Script:** doGet() / doPost() functions in apps-script.js

3.6 Configuration Files

File	Configuration
vite.config.js	Plugins: React + Tailwind; base: './' for relative asset paths
package.json	Scripts: dev (vite), build (vite build), start (node server.js), lint (eslint), preview (vite preview)
.npmrc	legacy-peer-deps=true — resolves peer dependency conflicts
.env / .env.production	VITE_SHEETS_URL pointing to the Google Apps Script Web App URL
.gitignore	Ignores node_modules, dist, .env* local, build, data/

4. Architecture Analysis

4.1 Overall Application Architecture

The project follows a **JAMstack-inspired architecture**:

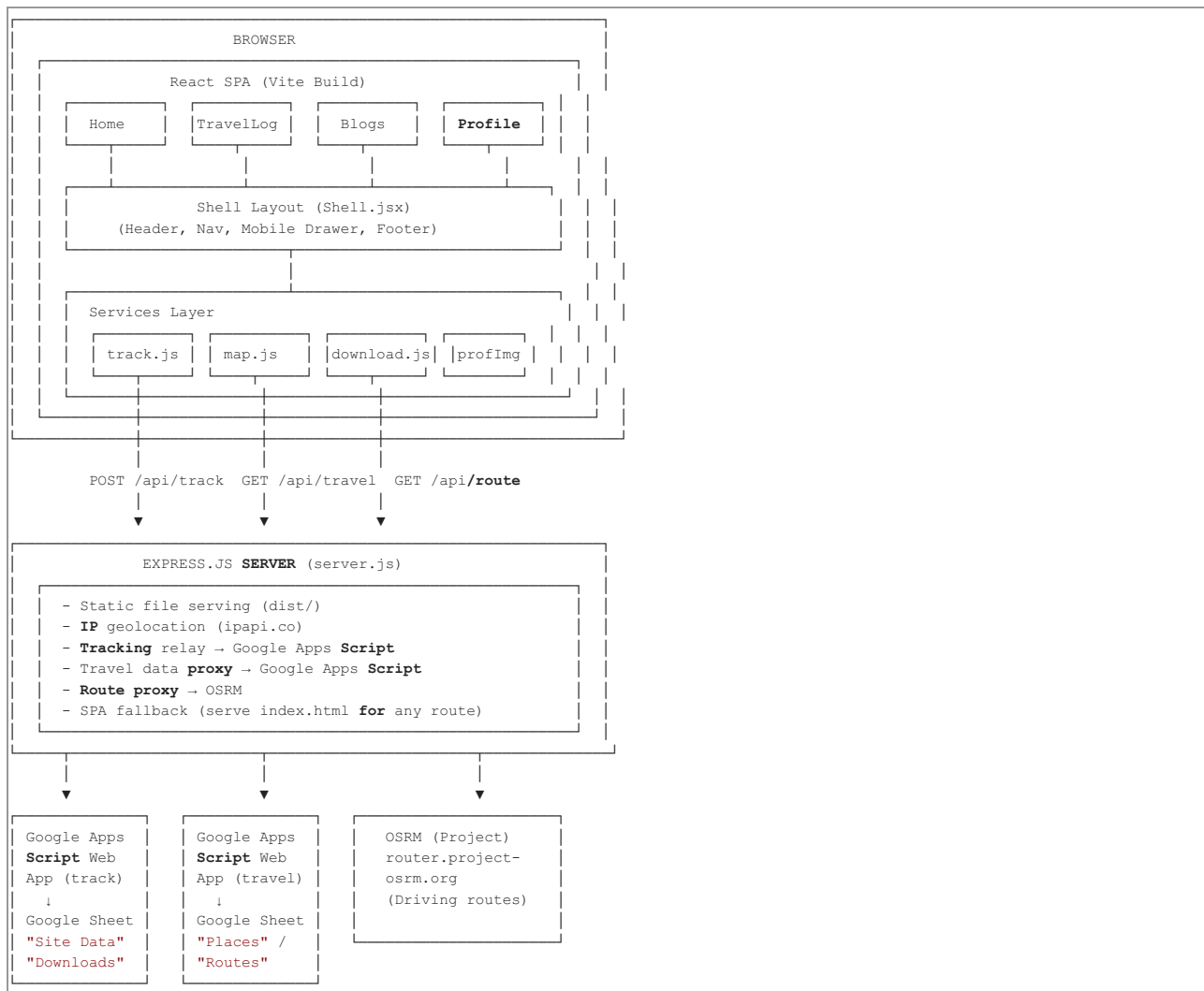
- **JavaScript** (React SPA frontend)
- **APIs** (Express backend, Google Apps Script, OSRM, ipapi.co)
- **Markup** (pre-rendered HTML shell with client-side rendering)

The frontend is a **React SPA** built with Vite. The backend is a thin **Express.js** middleware layer that:

1. Serves the built static frontend
2. Proxies tracking data to Google Apps Script
3. Proxies OSRM route requests (avoiding CORS)
4. Enriches tracking data with IP geolocation

Data persistence is outsourced to **Google Sheets** via a **Google Apps Script** Web App, acting as a serverless backend.

4.2 Architecture Diagram



4.3 Design Patterns Used

Pattern	Where	Description
Single Page Application (SPA)	Entire frontend	Client-side rendering with React Router
Proxy Pattern	server.js	Express proxies API calls to external services (GAS, OSRM)
Service Layer	src/services/	Separates UI from business/API logic
Container/Presentational	Pages (containers) ↔ Components (presentational)	Pages manage state, components render UI
Higher-Order Component (HOC)	BrowserRouter wrapping App	React Router's HOC pattern
Custom Hooks (intended)	src/hooks/ (empty)	Reserved for custom React hooks
Singleton	map.js legend control	Single legend instance per map
Cache-Aside	map.js localStorage caching	Route data cached with invalidation on places change

4.4 Frontend Architecture

The frontend is structured as:

- Entry** (main.jsx) — Mounts React app inside BrowserRouter
- Root** (App.jsx) — Global state (CV modal, timeout), route definitions, scroll restoration
- Layout** (Shell.jsx) — Fixed header, navigation, mobile drawer, profile lightbox
- Pages** — Route-specific content (Home, TravelLog, Blogs, Profile)
- Components** — Reusable modals (DownloadModal, TimeoutModal)
- Services** — API interaction and business logic

The component tree:

```
<BrowserRouter>
  <App>
    <ScrollToTop />
    <Shell>
      <Routes>
        <Route "/" → <Home> />
        <Route "/travel-log" → <TravelLog> />
        <Route "/blogs" → <Blogs> />
        <Route "/profile" → <Profile> />
      </Routes>
    </Shell>
    <DownloadModal />      (conditional on cvOpen state)
    <TimeoutModal />      (conditional on timedOut state)
  </App>
</BrowserRouter>
```

4.5 Backend Architecture

The Express server (server.js) is a thin API proxy layer with four responsibilities:

Route	Method	Purpose	Proxied To
/api/track	POST	Accept visitor tracking data, enrich with IP geo, relay to GAS	Google Apps Script (GET redirect)
/api/travel	GET	Fetch travel places and routes from Google Sheets	Google Apps Script (GET with action=travel)
/api/route	GET	Fetch driving route between two coordinates	OSRM (router.project-osm.org)
* (catch-all)	GET	SPA fallback—serve dist/index.html	Static file

4.6 Data Flow

Visitor Tracking Flow

```
Browser Event (pageview / click / retry)
  → trackVisitor(action) in track.js
  → POST /api/track (JSON payload: browser, device, OS, screen, etc.)
    → Server receives request
      → Server extracts IP from x-forwarded-for or remoteAddress
      → Server calls ipapi.co/{ip}/json/ for geolocation (if IP not localhost)
      → Server constructs URLSearchParams with all data
      → Server forwards to Google Apps Script via GET redirect-follow
      → GAS doGet() writes row to "Site Data" sheet
      → Server responds { ok: true }
```

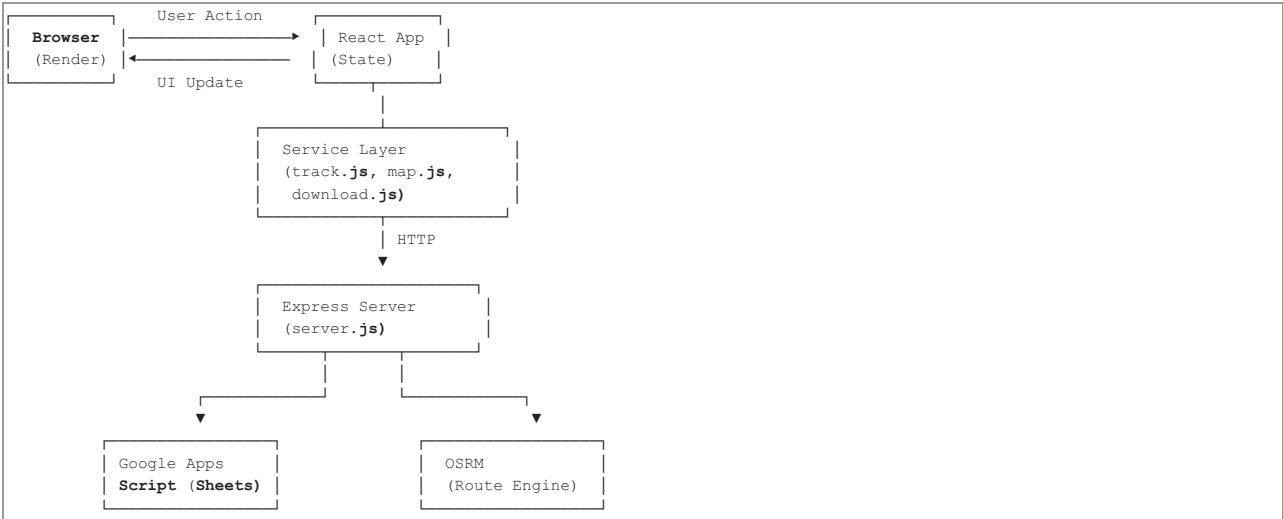
Travel Map Data Flow

```
TravelLog page mounts
  → GET /api/travel
    → Server relays to Google Apps Script with ?action=travel
    → GAS doGet() reads "Places" and "Routes" sheets
    → Returns JSON { places: [...], routes: [...] }
    → Server returns parsed JSON
  → Places shown immediately as markers on Leaflet map
  → fetchAllRoutes() called for each consecutive pair of places
    → For each pair (n to n+1):
      → Check localStorage cache → if cached, use it
      → If not cached: GET /api/route?from=lng,lat&to=lng,lat
      → Server proxies to OSRM
      → Returns coordinate array
    → Cache result in localStorage
  → Polylines rendered with animation
```

PDF Download Flow

```
User clicks "Download Data" or "Download My Data"
  → DownloadModal opens (name entry)
  → User enters name, clicks download
  → trackDataDownload(name) → POST /api/track (type: "Data Download")
  → downloadCV() creates an anchor element, triggers download of /Abilash-Data.pdf
  → Modal shows success state, closes after 800ms
```

4.7 Data Flow Diagram



4.8 Component / Module Relationships

```
App.jsx
├── trackVisitor() ← track.js
├── Shell.jsx
│   ├── profileImg ← profileImg.js (dynamic image resolver)
│   └── profile.jpeg (via img src)
├── Home.jsx
│   ├── Images: tea, batman, music, pet, god, bike
│   └── Framermotion animations
├── TravelLog.jsx
│   ├── L (leaflet) ← Map rendering
│   ├── FALLBACK_PLACES, renderLayers, fetchAllRoutes,
│   ├── DARK_TILES, TILE_OPTIONS, addLegend, markerType ← map.js
│   └── GET /api/travel ← Server proxy → Google Sheets
├── Blogs.jsx
│   └── profileImg ← profileImg.js
├── Profile.jsx
│   └── profileImg ← profileImg.js
├── DownloadModal.jsx
│   ├── trackDataDownload() ← track.js
│   └── downloadCV() ← download.js
└── TimeoutModal.jsx
    └── onRetry → trackVisitor('retry') + resetTimer
```

4.9 Communication Between Layers

Layer A	Layer B	Protocol	Data Format	Notes
React App	Express Server	HTTP (fetch)	JSON	Same origin, relative URLs (/api/*)
Express Server	Google Apps Script	HTTP (fetch)	URLSearchParams / Query Params	Follows 302 redirects manually
Express Server	OSRM	HTTP (fetch)	Query Params	Proxies driving route requests
Express Server	ipapi.co	HTTP (fetch)	JSON	IP geolocation enrichment
React App	Browser localStorage	JS API	JSON string	Route polyline caching
React App	Browser DOM	JS API	N/A	PDF download trigger

5. Source Code Analysis

5.1 Main Application Flow

Initialization Sequence

1. **Browser** loads `index.html`
2. **Vite-built** `main.jsx` executes:
 - Creates **React** root on `#root` div
 - Wraps `<App>` inside `<BrowserRouter>` from `react-router-dom`
 - Imports `index.css` (**Tailwind** + custom styles)
3. `App.jsx` renders:
 - `<ScrollToTop />` — resets scroll position on route change
 - `<Shell>` — persistent layout header/nav

- `<Routes>` — client-side page switching
- `<DownloadModal>` — globally accessible (via `cvOpen` state)
- `<TimeoutModal>` — global inactivity overlay

4. On mount: `trackVisitor('pageview')` fires → sends analytics
5. 60-second inactivity timer starts; resets on user interaction

Route Mapping

Path	Component	Description
<code>/</code>	Home	Landing page with hero, traits, explore panels, greeting overlay
<code>/travel-log</code>	TravelLog	Interactive Leaflet map with places and routes
<code>/blogs</code>	Blogs	Blog listing with filters and reading modal
<code>/profile</code>	Profile	Personal profile with status, links, interests

5.2 Routing System

The app uses **React Router DOM v6** with:

- **BrowserRouter** — HTML5 history API for clean URLs
- **Routes / Route** — Declarative route matching
- **Link** — Client-side navigation (used in Shell navigation)
- **useLocation** — Access current pathname (used in Shell active link highlighting, ScrollToTop, route transition animations)
- **AnimatePresence mode="wait"** — Page transition animations (fade in/out)

The server has a catch-all `GET *` route that always serves `dist/index.html` to support client-side routing on page refresh.

5.3 State Management

The project uses **React's built-in state management** (no Redux, Zustand, etc.):

Global State (App.jsx)

State	Type	Purpose
<code>cvOpen</code>	boolean	Controls <code>DownloadModal</code> visibility
<code>timedOut</code>	boolean	Controls <code>TimeoutModal</code> visibility
<code>lastActivity</code>	ref (number)	Timestamp of last user activity
<code>timerRef</code>	ref (interval)	Reference to the inactivity check interval

Page-Level State

Home.jsx:

- `greeting` (boolean) — Toggles "Say Hello" overlay

TravelLog.jsx:

- `places` (array) — Travel destinations (with fallback)
- `activePlace` (object/null) — Currently selected place
- `routes` (object/null) — Cached route polylines
- `loading` (boolean) — Route loading status
- `progress` ((done, total)) — Route fetch progress
- `error` (boolean) — Route fetch error state
- `showVisited` (boolean) — Toggle visited markers
- `showSmall` (boolean) — Toggle small stop markers

Blogs.jsx:

- `filter` (string) — Active tag filter ('all', 'Experience', 'life', 'update')
- `openPost` (object/null) — Currently open blog post in modal

Profile.jsx:

- `imgHover` (boolean) — Profile image hover state for "Available" tooltip

DownloadModal.jsx:

- `name` (string) — User's input name
- `submitting` (boolean) — Submission in progress
- `status` (string) — Feedback message

5.4 API Communication

All API calls use the **Fetch API** (native browser + Node.js 18+).

Frontend → Backend

```
// Track visitor (POST)
fetch('/api/track', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(payload),
  keepalive: true,
})

// Fetch travel data (GET)
fetch('/api/travel').then(r => r.json())

// Fetch OSRM route (GET)
fetch('/api/route?from=lng,lat&to=lng,lat').then(r => r.json())
```

Backend → External Services

The server uses `fetch()` with manual 302 redirect handling for Google Apps Script (which always redirects GET requests).

```
// GAS redirect-follow pattern
async function gasGet(params) {
  var url = GAS_URL + '?' + params.toString()
  var res = await fetch(url, { redirect: 'manual' })
  if (res.status >= 300 && res.status < 400) {
    var location = res.headers.get('location')
    // ... follow redirect manually
    var finalRes = await fetch(location)
    return finalRes
  }
  return res
}
```

5.5 Data Handling

Google Sheets Data Transformation (Apps Script)

Places sheet → JSON:

```
// Raw sheet row: [City, Country, Lat, Lng, Emoji, Date]
// → { city, country, lat, lng, emoji, date }
```

Routes sheet → JSON:

```
// Raw sheet row: [Name, Color, "lat,lng;lat,lng;..."]
// → { name, color, points: [[lat,lng], ...] }
```

Route Caching (localStorage)

The `map.js` module implements a client-side caching layer:

- **Cache key:** `travel_routes_v6`
- **Hash key:** `travel_places_hash` — used to detect sheet data changes
- **Cache entry:** `"{from_lat},{from_lng}|{to_lat},{to_lng}"` → coordinate array
- **Invalidation:** When place coordinates change, the entire cache is cleared
- **Batch fetching:** Routes are fetched in batches of 15 with a configurable delay

```
function invalidateCacheIfPlacesChanged(places) {
  var hash = places.map(p => p.lat + ',' + p.lng).join('|')
  var prev = localStorage.getItem(CACHE_HASH_KEY)
  if (prev !== hash) {
    localStorage.removeItem(CACHE_KEY)
    localStorage.setItem(CACHE_HASH_KEY, hash)
  }
}
```

5.6 Error Handling

Frontend

- **API calls** use `.catch()` to silently handle errors (no user-facing error messages for tracking)
- **Route fetching:** Error state shown with "Retry" button in TravelLog
- **Download modal:** Client-side validation for empty name
- **Route computation:** Falls back to straight-line (as-the-crow-flies) if OSRM fails

Backend (server.js)

- **Try/catch blocks** wrap all async operations
- **Tracking errors** are **logged** but never propagated to the client (always responds `{ ok: true }`)
- **Travel/Route errors** return empty data arrays instead of error responses
- **Graceful degradation:** If Google Apps Script is unreachable, places fallback to `[{ city: 'Coimbatore', ... }]`
- **IP geolocation timeout:** 3-second abort controller on `ipapi.co` call

Google Apps Script

- **Try/catch** in `doPost()` — returns `{ success: false, error: err.message }` on failure
- **Sheet creation** — creates "Downloads" or "Site Data" sheet if it doesn't exist

5.7 Authentication / Authorization

There is no authentication or authorization in this project. The site is fully public. The Google Apps Script Web App is deployed with "Anyone" access. The OSRM API is a public service. IP geolocation is fetched from a public API.

5.8 Important Modules Deep-Dive

map.js — Travel Map Service

This is the most complex module. Key features:

1. Marker System (SVG-based)

Three marker types, each rendered as inline SVG via Leaflet `divIcon`:

Type	Color	Size	Animation	Behavior
current	Green (#3bf66a)	36×36	Pulsing ring (CSS animation)	Always visible
visited	Blue (#3b82f6)	20×20	Static	Togglable
small	Red (#e60303f5)	16×16	Static	Togglable

2. Route Rendering System

- Consecutive places connected by polylines
- Real routes from OSRM when available, straight-line fallback otherwise
- Animated drawing using SVG `stroke-dashoffset` technique
- Color: Amber/yellow (#f7e309c2)

3. Caching Architecture

- Two-tier: memory (routes state) + localStorage (persistent)
- Hash-based invalidation
- Batch fetching (15 parallel requests) with progress callback

4. Legend Control

- Custom Leaflet control positioned bottom-left
- Shows color-coded legend for markers and routes

track.js — Visitor Tracking Service

Comprehensive client-side analytics:

Data Collected:

- Timestamp (date/time)
- Browser (Chrome, Firefox, Safari, Edge, Opera, Samsung Browser, Other)
- Device type (Desktop, Mobile, Tablet)
- Device brand (Samsung, Apple, Google, OnePlus, Xiaomi, etc.)
- Operating System (Windows, macOS, Linux, Android, iOS)
- Screen resolution
- Language
- Referrer URL
- Current URL path
- IP address + city/country/region (server-enriched)

Two tracking events:

1. `trackVisitor(action)` — Pageview and retry events
2. `trackDataDownload(name)` — PDF download events

download.js — PDF Download

A simple utility that creates a temporary `<a>` element pointing to `/Abilash-Data.pdf` and programmatically clicks it.

profileImg.js — Dynamic Image Resolver

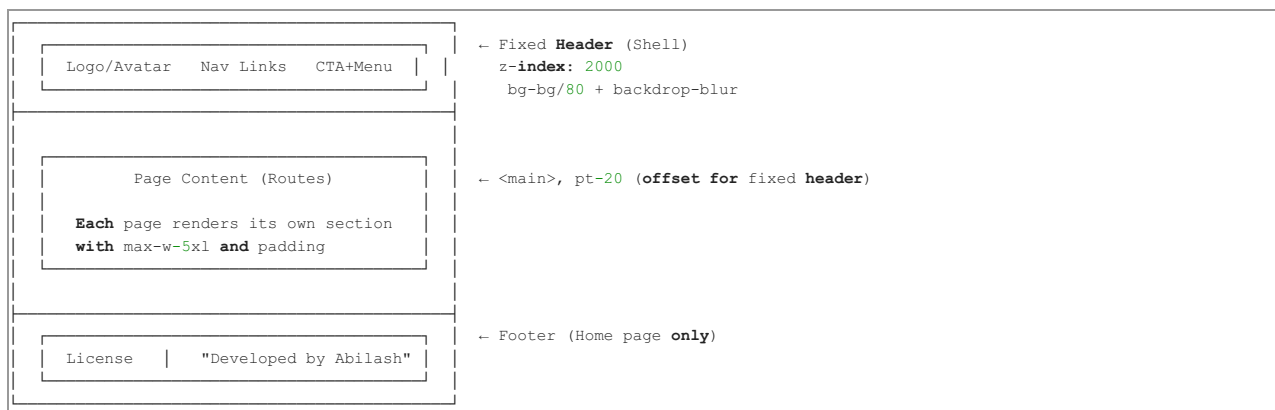
Uses Vite's `import.meta.glob` with `eager: true` to dynamically find any file matching `../images/profile.*` (jpg, jpeg, png, webp, etc.). This allows the profile image to change without updating import paths.

```
const profileModules = import.meta.glob('../images/profile.*', { eager: true })
const profileImg = Object.values(profileModules)[0]?.default || null
```

6. UI/UX Analysis

6.1 User Interface Structure

The UI follows a **single-column, content-centric layout**:



6.2 Pages / Screens

Home Page (/)

Sections:

1. **Hero** — Large "ABILASH" heading, subtitle, CTA buttons ("Download My Data", "Say Hello"), social links (IG, Chess, X), location badge
2. **About + Traits** — Split card layout: quote/about text on left, 6 trait cards (Tea Person, Im Batman, Music Addict, Pet, God, Motorcyclist) with images on right
3. **Explore** — 3 navigation cards (Profile, Travel Log, Blog) with hover effects
4. **Footer** — License (CC NC-ND-4.0), developer credit

Special overlays:

- **"Say Hello" overlay** — Animated greeting with wave emoji, particle effects, gradient text
- **Background glow** — Subtle radial gradient glow behind hero section

Travel Log (/travel-log)

Structure:

1. **Header** — Globe icon, "Travel Log" title, places count badge
2. **Map** — Full-width Leaflet map (CartoDB Dark Matter tiles) with toggle pills overlay
3. **Place list** — Grid of place cards showing emoji, city, country, date; click to fly to on map

Map features:

- Custom SVG markers (pulsing for current location)
- Animated route polylines
- "Visited" and "Stops" toggle buttons
- Loading progress bar
- Error state with retry button
- Custom dark-themed popups and zoom controls

Blogs (/blogs)

Structure:

1. **Header** — PenLine icon, "Blogs" title, post count
2. **Filter bar** — Tag pills: All, Experience, life, update
3. **Post list** — Clickable rows with tag badge, title, read time, date
4. **Reading modal** — Full-screen overlay with:
 - Custom yellow cursor highlighter
 - Tag, read time, date
 - Blog content
 - Author bio
 - "Done reading" button

Profile (/profile)

Structure:

1. **Hero card** — Profile image (with "Available" tooltip on hover), name, bio, "Currently somewhere..." status
2. **Status grid** — 4 traits: R-Status, Values, Profession, Passion
3. **Social Links** — Instagram, Snapchat, Twitter, Email with external link icons
4. **About Me** — Two paragraphs with interest tags (Love, Money, Travel, Music, Tea)
5. **Things I Enjoy** — 6 interest cards (Traveling, Minimalism, Chess, Cooking, Mountains, Love)

6.3 Components

Reusable Components

DownloadModal (src/components/DownloadModal.jsx)

- **Props:** open, onClose
- **State:** name, submitting, status
- Animated entry/exit with Framer Motion
- Form with name input, cancel/download buttons
- Success state with checkmark

- Tracks download event

TimeoutModal (`src/components/TimeoutModal.jsx`)

- Props: `onRetry`
- State: none (stateless)
- Animated entry with clock icon rotation
- "Try Again" button that calls `onRetry`
- Fixed full-screen overlay with blur

Layout Components

Shell (`src/layouts/Shell.jsx`)

- Fixed header with logo/avatar, desktop nav, actions
- Mobile hamburger → slide-in drawer (280px, right side)
- Profile image lightbox (full-screen)
- Page transition animations via `AnimatePresence`
- Active link detection via `useLocation()`

6.4 Design System

Color Palette

Token	Value	Usage
<code>bg</code>	<code>#09090b</code>	Page background
<code>bg-subtle</code>	<code>#111113</code>	Subtle background variant
<code>surface</code>	<code>#18181b</code>	Card/surface backgrounds
<code>surface-hover</code>	<code>#1f1f23</code>	Hover state for surfaces
<code>border</code>	<code>#27272a</code>	Default borders
<code>border-hover</code>	<code>#3f3f46</code>	Hover state for borders
<code>text</code>	<code>#fafafa</code>	Primary text
<code>text-secondary</code>	<code>#a1a1aa</code>	Secondary text
<code>text-tertiary</code>	<code>#71717a</code>	Tertiary/muted text
<code>text-quaternary</code>	<code>#52525b</code>	Lowest emphasis text
<code>accent</code>	<code>#3b82f6</code>	Primary accent (blue)
<code>accent-hover</code>	<code>#2563eb</code>	Accent hover state
<code>accent-glow</code>	<code>rgba(59,130,246,0.15)</code>	Accent glow effect

Note: Though the CSS defines `green`, `amber`, `red`, and `blue` tokens, they are all assigned the same value (`#3b82f6`).

Typography

Token	Family	Usage
<code>font-sans</code> / <code>font-display</code> / <code>font-spartan</code>	Gabarito	Primary UI font
<code>font-mono</code>	Gabarito	Monospace-styled text (badges, labels)
<code>Cedarville Cursive</code>	Google Font	Available but not used in current code

Font sizes range from 9px (mono labels) to 7xl (hero heading).

Animation System

Name	Type	Duration	Easing	Usage
<code>fadeIn</code>	Opacity	0.4s	<code>ease-out</code>	General entries
<code>scaleIn</code>	Opacity + Scale	0.3s	<code>cubic-bezier(0.16,1,0.3,1)</code>	Modal entries
<code>glow</code>	Opacity pulse	3s	<code>ease-in-out infinite</code>	Background glow
<code>float</code>	<code>TranslateY</code>	3s	<code>ease-in-out infinite</code>	Hover lift
Page transitions	Opacity + Y	0.2s	<code>cubic-bezier(0.16,1,0.3,1)</code>	Route changes

Custom CSS Layer

The project overrides Leaflet default styles to match the dark theme:

- Map background
- Popup wrapper and tip
- Zoom controls
- Custom scrollbar (5px, transparent track)

6.5 User Interactions

Interaction	Effect
Navigation click	Client-side route change with 200ms fade transition
Explore card hover	Accent gradient line at top, arrow slide-in, bottom accent bar scales in
Place card click (Travel)	Map flies to location with 1.5s duration
Blog post click	Modal opens with scale animation (0.25s)
Profile image hover	"Available" tooltip appears below
"Say Hello" click	Full-screen greeting overlay with wave animation and particles
Data download	Modal → name entry → download trigger + tracking
Inactivity (60s)	Timeout modal appears
Mobile hamburger	Right-side drawer slides in (0.35s)
Inline social links	External links open in new tabs

6.6 Responsive Behavior

Breakpoint	Behavior
Mobile (< 640px)	Single column layout, hamburger menu, smaller hero text, stacked trait cards
Tablet (640-768px)	2-column grids for traits/status
Desktop (> 768px)	Full multi-column layouts, visible nav bar, 3-column explore panels
Map	Height: 380px (mobile) → 480px (desktop)

The `Shell.jsx` navigation switches from inline links to a slide-in drawer at the `md: breakpoint` (768px).

6.7 Accessibility Considerations

Aspect	Implementation
ARIA labels	Mobile menu button has <code>aria-label="Toggle menu"</code>
Keyboard navigation	Enter key submits download form
Focus indicators	Input fields have <code>outline:none</code> with focus border/ring
Color contrast	Dark theme with high contrast text (<code>#fafafa</code> on <code>#09090b</code>)
User select	Globally disabled (<code>user-select: none</code>) except input/textarea/blog-content
Viewport meta	Standard responsive viewport tag
Semantic HTML	<code><header></code> , <code><main></code> , <code><nav></code> , <code><footer></code> elements used

Notable limitations:

- No `alt` text on trait images in `Home.jsx` (only `alt={t.name}` is provided, which is acceptable)
- No skip-to-content link
- No focus trapping in modals
- No reduced-motion media query support

7. Database Analysis

7.1 Database Structure

The project uses **Google Sheets** as its data store, interfaced through **Google Apps Script** Web App endpoints. There is no traditional SQL/NoSQL database.

7.2 Tables / Collections

Sheet: "Site Data" (apps-script-code.gs)

Column	Header	Description
A	Type	Event type ("Visitor" or "Data Download")
B	Name	Visitor name (for downloads) or empty
C	Action	Action type ("pageview", "retry", or empty)
D	Date	Date in DD/MM/YYYY format
E	Time	Time in HH:MM:SS format
F	Browser	Chrome, Firefox, Safari, etc.
G	Device	Desktop, Mobile, Tablet
H	OS	Windows, macOS, Linux, Android, iOS
I	Screen	Screen resolution (e.g. "1920x1080")
J	Language	Browser language
K	Referrer	Referrer URL or "Direct"
L	URL	Current page path

Sheet: "Downloads" (apps-script.js)

Column	Header	Description
A	No	Auto-incrementing number
B	Name	Visitor's entered name
C	Date	Download date
D	Time	Download time

Sheet: "Places" (apps-script.js)

Column	Header	Description
A	City	City name
B	Country	Country name
C	Lat	Latitude (number)
D	Lng	Longitude (number)
E	Emoji	Emoji representation
F	Date	Visit date string

Sheet: "Routes" (apps-script.js)

Column	Header	Description
A	Name	Route name
B	Color	Hex color string (e.g. "#a855f7")
C	Points	Semicolon-separated "lat,lng" pairs

7.3 Relationships

There are **no formal relationships** between sheets. Each sheet operates independently:

- **Places ↔ Routes**: Implicitly related by shared city names, no foreign keys
- **Site Data ↔ Downloads**: Independent tracking stores

7.4 Data Models

Places (JavaScript representation)

```
{
  city: "Coimbatore",    // String
  country: "India",      // String
  lat: 11.0168,          // Number (float)
  lng: 76.9558,          // Number (float)
  emoji: "🏠",            // String (emoji)
  date: "Home Base"      // String (date or label)
}
```

Routes

```
{
  name: "Chennai to Bangalore", // String
  color: "#a855f7",             // String (hex)
  points: [[12.97, 77.59], ...] // Array of [lat, lng] pairs
}
```

Visitor Tracking Payload

```
{
  type: "Visitor",           // "Visitor" | "Data Download"
  action: "pageview",        // "pageview" | "retry" | ""
  name: "",                  // Visitor name (for downloads)
  date: "29/07/2026",        // DD/MM/YYYY
  time: "14:30:00",          // HH:MM:SS
  browser: "Chrome",         // Browser name
  device: "Desktop",         // Desktop | Mobile | Tablet
  brand: "Unknown",          // Device brand
  os: "Windows",             // OS name
  screen: "1920x1080",       // Width x Height
  language: "en-US",         // Navigator language
  referrer: "Direct",        // Referrer or "Direct"
  url: "/travel-log",        // Current path
  ip: "203.0.113.0",         // (enriched by server)
  city: "Mumbai",            // (enriched by server via ipapi.co)
  country: "India",          // (enriched by server)
  region: "Maharashtra"     // (enriched by server)
}
```

7.5 CRUD Operations

Since the data is stored in Google Sheets and accessed via Google Apps Script, CRUD operations are limited:

Operation	Implementation	Sheet
Create (Tracking)	sheet.appendRow([...])	Site Data / Downloads
Read (Places)	sheet.getDataRange().getValues() → JSON mapping Places	
Read (Routes)	sheet.getDataRange().getValues() → JSON mapping Routes	
Update	Not implemented	N/A
Delete	Not implemented	N/A

The project is **write-once / read-many** for tracking, and **read-only** for travel data.

7.6 Data Flow

Write Path (Tracking)

```
Browser Event → track.js → POST /api/track → Google Apps Script → sheet.appendRow()
```

Read Path (Travel)

```
TravelLog mount → GET /api/travel → Google Apps Script doGet() → sheet.getDataRange().getValues()
→ JSON response → Leaflet map markers + route polylines
```

8. API Documentation

8.1 Track Endpoint

```
POST /api/track
```

Purpose: Accept visitor tracking data, enrich with IP geolocation, relay to Google Sheets.

Request Body (JSON):

Field	Type	Required	Description
type	string	Yes	"Visitor" or "Data Download"
action	string	No	"pageview", "retry", or empty
name	string	No	Visitor name (download events only)
date	string	Yes	DD/MM/YYYY
time	string	Yes	HH:MM:SS
browser	string	Yes	Browser name
device	string	Yes	Desktop/Mobile/Tablet
brand	string	Yes	Device brand
os	string	Yes	Operating System
screen	string	Yes	Screen resolution
language	string	Yes	Browser language
referrer	string	Yes	Referrer URL or "Direct"
url	string	Yes	Current page path

Response:

```
{ "ok": true }
```

Server-side processing:

1. Extracts IP from x-forwarded-for header or socket.remoteAddress
2. Ignores localhost IPs (::1, 127.0.0.1)
3. Calls `ipapi.co/{ip}/json/` for geolocation (3s timeout)
4. Constructs `URLSearchParams` with all fields including IP, city, country, region
5. Forwards to Google Apps Script via GET request with manual 302 redirect following
6. Returns `{ ok: true }` immediately (does not wait for GAS response)

8.2 Travel Endpoint

```
GET /api/travel
```

Purpose: Fetch travel places and routes from Google Sheets.

Query Parameters: None

Response:

```
{
  "places": [
    {
      "city": "Coimbatore",
      "country": "India",
      "lat": 11.0168,
      "lng": 76.9558,
      "emoji": "🇮🇳",
      "date": "Home Base"
    }
  ],
  "routes": [
    {
      "name": "Route Name",
      "color": "#a855f7",
      "points": [[lat, lng], [lat, lng]]
    }
  ],
  "_debug": {
    "allSheets": ["Places", "Routes"],
    "placesFound": true,
    "routesFound": true,
    "placesCount": 10,
    "routesCount": 3
  }
}
```

Error response: `{ "places": [] }`

Server-side processing:

1. Forwards request to Google Apps Script with `action=travel` parameter
2. Follows GAS 302 redirect manually
3. Attempts JSON parse; returns `{ places: [] }` on failure

8.3 Route Endpoint

```
GET /api/route
```

Purpose: Fetch driving route between two coordinates via OSRM.

Query Parameters:

Parameter	Type	Required	Description
from	string	Yes	Coordinates in "lng,lat" format
to	string	Yes	Coordinates in "lng,lat" format

Response (success):

```
{
  "coords": [[lat, lng], [lat, lng], ...]
}
```

Response (error):

```
{ "coords": null }
```

Response (missing params):

```
{ "error": "missing from/to" }
```

Upstream API: OSRM (Open Source Routing Machine) at <https://router.project-osrm.org/route/v1/driving/{from};{to}?overview=simplified&geometries=geojson&steps=false>

8.4 Google Apps Script API

Deployment URL: <https://script.google.com/macros/s/AKfycbwhnLnBeAyWammouZHO-z41b5fGDG-MNEZtMOPoPYKAs218QokxLzkOVzrW5fiUN3gW5g/exec>

doGet(e)

Purpose: Handle GET requests. Supports two modes:

Mode 1: Travel data (?action=travel)

- Reads "Places" and "Routes" sheets
- Returns JSON { places: [...], routes: [...], _debug: {...} }

Mode 2: Tracking (apps-script-code.gs)

- Reads query parameters: type, name, action, date, time, browser, device, os, screen, language, referrer, url
- Appends row to "Site Data" sheet
- Returns "ok"

doPost(e) — Tracking (apps-script-code.gs)

Purpose: Handle POST requests for tracking data.

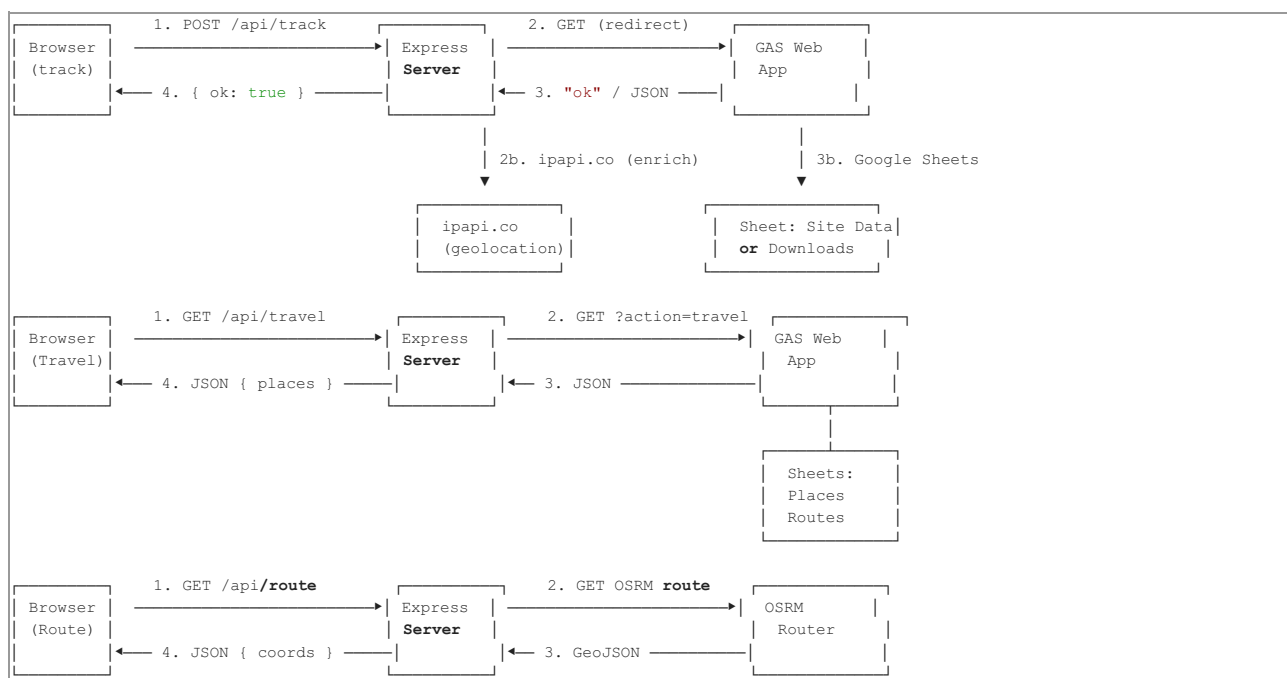
- Parses e.parameter
- Appends row to "Site Data" sheet
- Returns "ok"

doPost(e) — Downloads (apps-script.js)

Purpose: Handle POST requests for download tracking.

- Expects JSON body: { name, date, time }
- Appends row to "Downloads" sheet with auto-incrementing number
- Returns { success: true } or { success: false, error: message }

8.5 API Workflow Diagram



9. Dependencies Report

9.1 Production Dependencies

Package	Version	Purpose	Used Where
react	^18.2.0	UI component library for building the SPA	All JSX files
react-dom	^18.2.0	React DOM renderer	main.jsx
react-router-dom	^6.20.0	Client-side routing with HTML5 history API	App.jsx, Shell.jsx
framer-motion	^12.42.2	Animation library for page transitions, modals, overlays	Shell.jsx, Home.jsx, TravelLog.jsx, Blogs.jsx, Profile.jsx, DownloadModal.jsx, TimeoutModal.jsx
leaflet	^1.9.4	Interactive map library	TravelLog.jsx, map.js
lucide-react	^1.25.0	SVG icon components (MapPin, PenLine, User, Globe, Download, X, etc.)	Home.jsx, TravelLog.jsx, Blogs.jsx, Profile.jsx, Shell.jsx, DownloadModal.jsx, TimeoutModal.jsx
@tailwindcss/vite	^4.3.3	Tailwind CSS Vite integration plugin	vite.config.js
tailwindcss	^4.3.3	Utility-first CSS framework	index.css
express	^4.22.2	HTTP server framework for API proxy and static files	server.js
serve	^14.2.4	Static file serving (alternative to custom Express)	Listed in package.json but not directly used in code
jspdf	^4.2.1	PDF generation library	Listed in package.json but not actually used in source code
pdf-lib	^1.17.1	PDF manipulation library	Listed in package.json but not actually used in source code

9.2 Development Dependencies

Package	Version	Purpose	Used Where
@types/react	^18.2.0	TypeScript type definitions for React	Development (type checking if used)
@types/react-dom	^18.2.0	TypeScript type definitions for ReactDOM	Development (type checking if used)
@vitejs/plugin-react	^4.2.0	Vite plugin for React Fast Refresh	vite.config.js
vite	^5.0.0	Build tool and dev server with HMR	vite.config.js, package.json scripts

Note on unused dependencies: jspdf and pdf-lib are listed in package.json but not imported or used in any source file. They may be remnants of an earlier implementation or planned features.

10. Feature-by-Feature Explanation

10.1 Home / Landing Page

Purpose: First impression of the personal brand — introduces Abilash, showcases personality traits, and provides navigation to all sections.

Related Files:

- src/pages/Home.jsx
- src/layouts/Shell.jsx (navigation)
- src/services/profileImg.js (avatar)
- src/images/* (trait images)

Implementation Overview:

- Uses Framer Motion stagger variants to animate content on mount
- Three sections: Hero (name, tagline, CTA buttons, social links), About + Traits (6 personality trait cards with images), Explore (3 navigation panel cards)
- "Say Hello" button triggers a full-screen animated greeting overlay with particle effects
- Background glow effect using radial gradients

User Workflow:

1. User visits /
2. Hero section animates in with staggered text
3. User can click "Download My Data" → opens download modal
4. User can click "Say Hello" → playful greeting overlay appears
5. User can click any trait card (visual only)
6. User can click "Explore" panels → navigates to Profile, Travel Log, or Blogs

10.2 Travel Log Interactive Map

Purpose: Visually showcase all travel destinations on an interactive world map with route animations.

Related Files:

- src/pages/TravelLog.jsx
- src/services/map.js
- server.js (/api/travel, /api/route)
- apps-script.js (Google Apps Script data source)

Implementation Overview:

- Leaflet map with CartoDB Dark Matter tiles
- Three marker types: Current (green, pulsating), Visited (blue), Small stops (red)
- Routes computed via OSRM API, cached in localStorage
- Animated route drawing using SVG stroke-dashoffset
- Toggle filters for Visited and Stops markers
- Place cards below map for quick navigation (flyTo)
- Loading progress bar and error retry UI

User Workflow:

1. User navigates to /travel-log
2. Map loads centered on last place (first in array)

3. Places fetched from Google Sheets via `/api/travel`
4. Markers appear immediately; routes load in background
5. User can click toggle pills to show/hide Visited/Stops markers
6. User can click any place card → map flies to that location
7. User can interact with the map (pan, zoom, click markers for popups)

10.3 Blog Reader

Purpose: Display blog posts with filtering and a reading experience.

Related Files:

- `src/pages/Blogs.jsx`

Implementation Overview:

- Static blog data defined as an array of post objects (currently placeholder content)
- Tag-based filtering (all, Experience, life, update) with color-coded badges
- Click-to-read opens a full-screen modal with custom cursor highlighter
- Modal includes tag, reading time, date, full content, author bio
- "Done reading" button or backdrop click to close

User Workflow:

1. User navigates to `/blogs`
2. Blog list displays with tag badges
3. User can filter posts by tag
4. User clicks a post → reading modal opens with scale animation
5. Custom yellow cursor appears over content area
6. User reads, then clicks "Done reading" → modal closes

10.4 Profile Page

Purpose: Personal profile page displaying status, social links, biography, and interests.

Related Files:

- `src/pages/Profile.jsx`
- `src/services/profileImg.js`

Implementation Overview:

- Hero card with profile image (shows "Available" tooltip on hover, green status dot)
- Status grid (R-Status, Values, Profession, Passion)
- Social Links grid (Instagram, Snapchat, Twitter, Email)
- About Me section with descriptive paragraphs and interest tags
- Things I Enjoy grid (6 interest items)
- Staggered entry animation

User Workflow:

1. User navigates to `/profile`
2. Cards animate in with staggered Framer Motion
3. User can hover profile image → "Available" tooltip appears
4. User can click social links → opens in new tab
5. User reads about the developer

10.5 CV/Data Download

Purpose: Allow visitors to download a portfolio/data PDF while capturing their name for analytics.

Related Files:

- `src/components/DownloadModal.jsx`
- `src/services/download.js`
- `src/services/track.js`
- `public/Abilash-Data.pdf`
- `apps-script.js` (download tracking in Google Sheets)

Implementation Overview:

- Modal triggered from Shell header button, Home page CTA, or mobile drawer
- Form requires name input
- On submit: tracks event via `trackDataDownload()`, triggers PDF download
- Success state shown briefly before auto-closing
- PDF is a static file served from `public/Abilash-Data.pdf`

User Workflow:

1. User clicks "Data" (header), "Download My Data" (home), or "Download Data" (mobile)
2. DownloadModal opens with animated entry
3. User enters their name (required field highlighted with * and accent color)
4. User clicks "Download PDF" (or presses Enter)
5. PDF downloads to browser, download tracked to Google Sheets
6. Modal shows "Done" checkmark, closes after 800ms

10.6 Visitor Tracking

Purpose: Collect analytics about site visitors for the site owner.

Related Files:

- `src/services/track.js`
- `server.js (/api/track)`
- `apps-script-code.gs / apps-script.js`

Implementation Overview:

- `trackVisitor()` called on page load and timeout retry
- Collects: browser, device, brand, OS, screen resolution, language, referrer, URL
- Server enriches with IP geolocation from `ipapi.co`
- Data sent to Google Sheets via Google Apps Script
- `keepalive: true` ensures request completes even on page navigation

User Workflow (invisible to user):

1. On any page load → `trackVisitor('pageview')` fires
2. `track.js` constructs payload with browser/device/OS metadata
3. POST request to `/api/track` with JSON body
4. Server adds IP, city, country, region
5. Data forwarded to Google Sheets (Site Data sheet)
6. On CV download → `trackDataDownload(name)` fires with similar flow

10.7 Session Timeout Management

Purpose: Detect visitor inactivity and prompt re-engagement.

Related Files:

- `src/App.jsx` (timer logic)
- `src/components/TimeoutModal.jsx`

Implementation Overview:

- 60-second inactivity timer (resets on mousedown, keydown, scroll, touchstart)
- Timer checks every 5 seconds if `Date.now() - lastActivity >= 60000`
- On timeout: `timedOut` state → `TimeoutModal` appears
- "Try Again" button calls `trackVisitor('retry')`, resets timer, hides modal
- Note: The modal does not actually navigate; its label says "It will navigate to website" but no navigation logic exists in `onRetry`

User Workflow:

1. User stops interacting with the page for 60 seconds
2. `TimeoutModal` appears with animated clock icon
3. User clicks "Try Again"
4. Retry event tracked, modal dismissed
5. User can continue browsing

11. Code Quality Review

11.1 Code Organization

Strengths:

- Clear separation of concerns: `pages / components / layouts / services`
- Consistent file naming conventions (PascalCase for components, camelCase for services)
- Each service module has a single responsibility
- Images organized in a dedicated directory

Observations:

- `src/hooks/` and `src/utils/` and `src/assets/` are empty — planned but unused
- Some component logic could be extracted into custom hooks (e.g., the timeout logic in `App.jsx`)
- Inline styles mixed with Tailwind classes (e.g., `style={{ boxShadow: "... " }}`)

11.2 Maintainability

Strengths:

- Small, focused files (max ~367 lines for `Home.jsx`, ~245 for `Blogs.jsx`)
- Services abstract API logic from components
- Vite build produces optimized output with content hashing
- No complex state management library to learn

Concerns:

- `map.js` is 237 lines with multiple concerns (markers, routes, caching, legend) — could be split
- `apps-script.js` and `apps-script-code.gs` are similar but not identical — potential confusion
- Hardcoded Google Apps Script URL in `server.js` and `.env` files
- Blog content is hardcoded in `Blogs.jsx` rather than fetched from a data source
- Magic numbers (60 * 1000, 800ms delays, etc.) scattered in code

11.3 Scalability

Limitations:

- Google Sheets as a database will not scale beyond ~10,000 rows (Google Sheets limit)

- OSM caching in localStorage limited to ~5-10MB
- No pagination for blog posts or travel places
- No CDN or caching layer for static assets
- Google Apps Script has a 6-minute execution timeout and daily quotas

Positives:

- Serverless architecture (Google Sheets / Apps Script) eliminates server management
- Client-side caching reduces API calls
- SPA architecture allows easy addition of new pages/routes

11.4 Performance Considerations

Strengths:

- Vite build produces minified, code-split bundles
- Leaflet with `preferCanvas: true` for better rendering performance
- Route caching in localStorage avoids redundant API calls
- Batch fetching of routes (15 parallel requests)
- `keepalive: true` on tracking fetch

Concerns:

- No lazy loading for route components (all bundled in a single JS file)
- Images are not optimized (no WebP conversion, no lazy loading `loading="lazy"`)
- No memoization (`useMemo/useCallback`) on expensive computations
- Framer Motion might cause layout thrashing on low-end devices
- The Leaflet map re-renders all layers on any state change (`showVisited/showSmall` toggle)

11.5 Security Considerations

Strengths:

- No authentication tokens or secrets exposed in frontend code
- API proxy prevents direct exposure of Google Apps Script URL to CORS issues
- IP geolocation is a server-side operation (not client-facing)
- `noreferrer` on external links prevents referrer leakage

Concerns:

- Google Apps Script Web App URL is exposed in `.env` files committed to git (though `.env` is gitignored, `.env.production` is NOT in `.gitignore`)
- No rate limiting on `/api/track` endpoint
- No input sanitization (though only string data is collected)
- `ipapi.co` API key not rate-limited (free tier has limits)
- The server has no CORS configuration — relies on same-origin only

11.6 Best Practices Followed

Practice	Status	Notes
ES6+ syntax	✓	Arrow functions, <code>const/let</code> , template literals, destructuring
Semantic HTML	✓	<code><header></code> , <code><main></code> , <code><nav></code> , <code><footer></code> , <code><h1></code> – <code><h6></code>
Functional components	✓	No class components
Hooks	✓	<code>useState</code> , <code>useEffect</code> , <code>useRef</code> , <code>useCallback</code>
Cleanup in effects	✓	Event listeners removed, timers cleared
SPA fallback	✓	Express catch-all serves <code>index.html</code>
Environment variables	✓	<code>VITE_SHEETS_URL</code> via <code>.env</code> files
Error boundaries	✗	No React error boundaries implemented
PropTypes / TypeScript	✗	No type checking
Unit tests	✗	No test framework or test files
Accessibility		Partial ARIA labels, keyboard support present but limited
Code splitting	✗	No <code>React.lazy</code> or dynamic imports
Responsive design	✓	Mobile-first with Tailwind breakpoints

12. Learning Guide

12.1 What Should Be Learned First

To understand this project, a developer should be familiar with:

1. **JavaScript ES6+** — arrow functions, template literals, destructuring, modules, promises, `async/await`
2. **React.js** — components, props, state, hooks (`useState`, `useEffect`, `useRef`, `useCallback`)
3. **React Router v6** — `BrowserRouter`, `Routes`, `Route`, `Link`, `useLocation`
4. **CSS** — Flexbox, Grid, custom properties, responsive design
5. **Tailwind CSS** — utility classes, custom theme configuration
6. **Node.js / Express** — basic server setup, routing, middleware
7. **Google Apps Script** — Web App deployment, `SpreadsheetApp`, `ContentService`
8. **Leaflet.js** — map initialization, tile layers, markers, popups, controls

12.2 How the Project Works Internally

High-level flow:

1. **Build time:** Vite bundles React components, CSS, and images into static files in `dist/`
2. **Runtime (development):** Vite dev server with HMR
3. **Runtime (production):** Express serves static files and API proxying
4. **Data persistence:** Google Sheets via Google Apps Script
5. **Routing:** React Router handles client-side navigation; Express SPA fallback handles refreshes

Data never flows through a traditional database — everything goes through Google Sheets.

12.3 Important Files to Study First

In suggested order:

Order	File	Why
1	<code>src/main.jsx</code>	Entry point — learn how React app is mounted
2	<code>src/App.jsx</code>	Root component — understand routing, global state, page structure
3	<code>src/layouts/Shell.jsx</code>	Main layout — navigation, mobile drawer, lightbox
4	<code>src/pages/Home.jsx</code>	Complex page — animations, sections, trait display
5	<code>src/pages/TravelLog.jsx</code>	Most complex page — Leaflet map, async data, progress tracking
6	<code>src/services/map.js</code>	Core service — marker system, route rendering, caching layer
7	<code>src/services/track.js</code>	Analytics — user agent parsing, data collection
8	<code>server.js</code>	Backend — API proxy, IP enrichment, SPA fallback
9	<code>apps-script.js</code>	Serverless backend — Google Sheets CRUD
10	<code>vite.config.js</code>	Build configuration — plugins, base path

12.4 Suggested Learning Order

Week 1-2: React Fundamentals

1. Understand `main.jsx` → `App.jsx` → routing
2. Study how props and state flow through components
3. Learn Framer Motion basics (`AnimatePresence`, `motion.div`, variants)

Week 3: Styling

4. Study `index.css` — Tailwind theme configuration
5. Understand the custom CSS layer (Leaflet overrides, animations)
6. Explore how Tailwind utility classes are used in components

Week 4: Advanced Features

7. Deep-dive into `map.js` — SVG markers, Leaflet integration, caching
8. Study `track.js` — user agent parsing, async data collection
9. Understand the Express proxy pattern

Week 5: Full Stack

10. Study `server.js` — how the Express server works
11. Deploy the Google Apps Script to see the backend in action
12. Understand the complete data flow from browser → server → GAS → Sheets

Week 6: Build & Deploy

13. Run `npm run build` and inspect `dist/` output
14. Deploy to a platform like Render or Railway
15. Configure environment variables for production

13. Developer Documentation

13.1 Installation Requirements

- Node.js >= 18 (for native `fetch` API and ESM modules)
- npm >= 9
- Modern web browser for development
- Git for version control
- (Optional) Google account for Apps Script deployment

13.2 Setup Explanation

Clone & Install

```
git clone <repository-url>
cd my-personal-website
npm install
```

The `.npmrc` file sets `legacy-peer-deps=true` to handle potential peer dependency conflicts.

Environment Configuration

Copy the example environment file:

```
cp .env.example .env
```

Edit `.env` with your Google Apps Script URL (or leave empty to fall back to server-side handling).

Google Apps Script Deployment

1. Open `https://script.google.com/`
2. Create a new project
3. Paste contents of `apps-script.js` (or `apps-script-code.gs`)
4. Create a Google Sheet with sheets named "Places", "Routes" (for travel), and optionally "Site Data", "Downloads" (for tracking)
5. Deploy as Web App:
 - Execute as: Me
 - Who has access: Anyone
6. Copy the deployment URL into `.env` as `VITE_SHEETS_URL`
7. Also update the `GAS_URL` constant in `server.js`

13.3 Running Instructions

Development Mode

```
npm run dev
```

Starts Vite dev server (default: `http://localhost:5173`) with HMR.

Production Build

```
npm run build
```

Outputs to `dist/` directory.

Production Server

```
npm run build
npm start
```

Starts Express server (default: `http://localhost:3000`):

- Serves static files from `dist/`
- API endpoints at `/api/*`
- SPA fallback for all other routes

Preview Build

```
npm run build
npm run preview
```

Starts Vite preview server for the built output.

Linting

```
npm run lint
```

Runs ESLint on the project.

13.4 Environment Variables Explanation

Variable	Required	Default	Description
<code>VITE_SHEETS_URL</code>	No (dev) / Yes (prod)	""	Google Apps Script Web App URL. Used by the frontend if provided; otherwise falls back to server proxy (<code>/api/cv-download</code>)
<code>PORT</code>	No	3000	Port for the Express production server

Files:

- `.env` — Development environment (gitignored)
- `.env.production` — Production environment (committed to git)
- `.env.example` — Template with documentation (committed to git)

The `VITE_SHEETS_URL` is passed to the frontend at build time via Vite's `import.meta.env.VITE_*` convention.

13.5 Deployment Overview

Option 1: Render (Current)

The project appears to be deployed on **Render** based on:

- `.gitignore` has `# generated data` (stored via GitHub API on Render)
- `server.js` serves as the production entry point

Deployment steps for Render:

1. Connect GitHub repository to Render
2. Set build command: `npm run build`
3. Set start command: `npm start`
4. Add environment variable: `VITE_SHEETS_URL=<GAS_URL>`
5. Deploy

Option 2: Any Node.js Hosting

Compatible with:

- Railway
- Vercel (with serverless functions adaptation)
- Heroku

- DigitalOcean App Platform
- AWS Elastic Beanstalk

Option 3: Static Hosting (Frontend Only)

If deploying frontend only (without Express backend):

1. Build with `npm run build`
2. Deploy `dist/` to: Netlify, Vercel, GitHub Pages, Cloudflare Pages

Note: API features (tracking, travel data, routes) will not work without the Express proxy.

Important Notes for Deployment

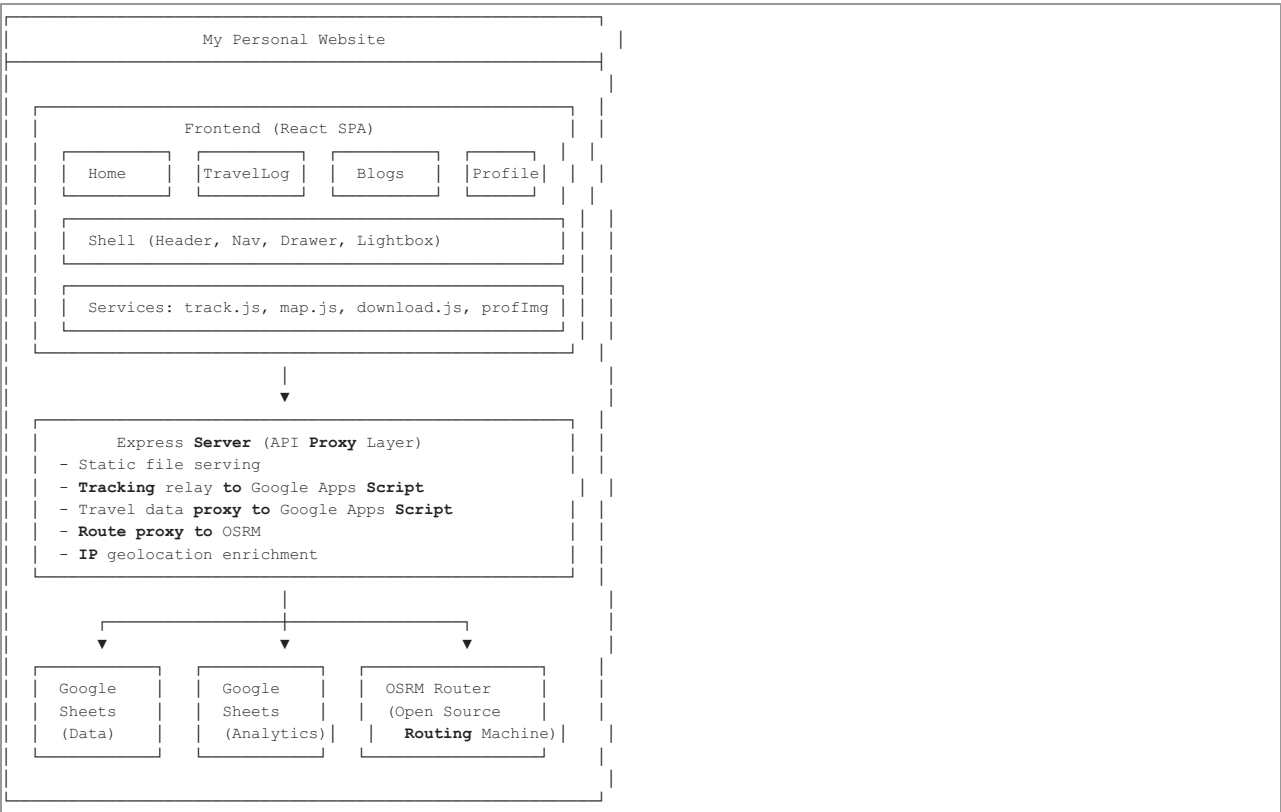
- The Google Apps Script Web App must be deployed with "Anyone" access
- CORS is handled by same-origin Express proxy — no CORS headers needed
- The server must handle SPA routing (already implemented via `app.get('*', ...)`)
- No database setup required (Google Sheets handles all persistence)
- The `VITE_SHEETS_URL` only needs to be set if the frontend directly calls GAS; the current implementation routes everything through the Express proxy

14. Final Summary

14.1 Project Strengths

Strength	Description
Clean Architecture	Well-organized separation of pages, components, layouts, and services
Modern Tech Stack	React 18, Vite 5, Tailwind v4, Framer Motion — all current industry standards
No-Code Backend	Google Sheets as a database eliminates server-side persistence concerns
Rich Interactivity	Animated page transitions, interactive map with custom SVG markers, modal systems
Analytics Built-in	Custom visitor tracking without third-party tools
Dark Theme	Visually appealing dark UI with consistent design language
Responsive	Fully responsive design with mobile navigation drawer
Serverless Ready	Can be deployed as static site or with thin Express layer

14.2 Current Architecture Summary



14.3 Key Technologies Learned from This Project

Technology	What It Teaches
React + Vite	Modern frontend tooling, Fast Refresh, build optimization
React Router v6	Client-side routing, <code>useLocation</code> , link navigation
Framer Motion	Declarative animations, <code>AnimatePresence</code> , <code>staggered children</code>
Tailwind CSS v4	Utility-first CSS, custom theme configuration, <code>@theme</code> directive
Leaflet	Interactive maps, custom SVG markers, tile layers, popups

Express.js	API proxying, static file serving, SPA fallback routing
Google Apps Script	Serverless functions, Google Sheets integration, Web App deployment
OSRM	Open Source Routing, driving directions, GeoJSON
localStorage	Client-side caching, cache invalidation strategies
Visitor Analytics	User agent parsing, IP geolocation, tracking infrastructure

14.4 Overall Evaluation

Rating: Good — Well-structured personal portfolio project

The project successfully delivers on its goals: creating an attractive personal website that showcases the developer's personality, travel history, and professional profile. The architecture is clean and follows modern web development best practices.

What works well:

- The separation of frontend (React SPA) and backend (Express proxy) is clear and maintainable
- The use of Google Sheets as a zero-configuration database is clever for a personal project
- The interactive map with custom SVG markers and animated routes is impressive
- The dark theme is consistent and visually polished
- No unnecessary complexity — the tech stack is appropriate for the project's scope

Areas for potential future improvement:

- Adding lazy loading for routes/pages
- Implementing error boundaries
- Adding proper test coverage
- Replacing hardcoded blog data with a CMS or markdown files
- Optimizing images with modern formats and lazy loading
- Implementing rate limiting for the tracking endpoint
- Adding proper TypeScript support

Overall: A solid, well-architected personal portfolio website that effectively demonstrates React, Vite, Leaflet, Express, and Google Apps Script integration. Suitable as a reference project for learning modern full-stack web development with a serverless data layer.

Appendix A: Complete File Inventory

#	File Path	Lines	Purpose
1	index.html	16	HTML entry point with font loading
2	vite.config.js	8	Vite build configuration
3	server.js	141	Express production server
4	package.json	33	Project manifest
5	.env / .env.example / .env.production	1/3/1	Environment configuration
6	.npmrc	1	npm peer dependency config
7	.gitignore	25	Git ignore rules
8	apps-script.js	124	Google Apps Script (full)
9	apps-script-code.gs	51	Google Apps Script (simple)
10	src/main.jsx	11	React DOM entry
11	src/App.jsx	78	Root component with routing + global state
12	src/index.css	265	Tailwind config + custom styles
13	src/layouts/Shell.jsx	236	Main layout with header, nav, lightbox
14	src/pages/Home.jsx	367	Landing page
15	src/pages/TravelLog.jsx	232	Interactive map page
16	src/pages/Blogs.jsx	245	Blog listing + reader
17	src/pages/Profile.jsx	241	Personal profile page
18	src/components/DownloadModal.jsx	121	PDF download prompt
19	src/components/TimeoutModal.jsx	49	Session timeout overlay
20	src/services/track.js	115	Visitor tracking service
21	src/services/map.js	237	Leaflet map utilities
22	src/services/download.js	8	PDF download helper
23	src/services/profileImg.js	4	Dynamic profile image resolver

Appendix B: npm Scripts Reference

Script	Command	Description
npm run dev	vite	Start development server with HMR
npm run build	vite build	Build for production
npm start	node server.js	Run production Express server
npm run lint	eslint .	Run ESLint on all files
npm run preview	vite preview	Preview production build locally